



4

Optimization Problems in Fog and Edge Computing

**Dr. B.V. Baiju,
SCOPE,
Assistant Professor,
VIT_Vellore**

Preliminaries

An optimization problem is generally defined by the following

- A **list of variables** $\mathbf{x} = (x_1, \dots, x_n)$.
 - **Represent the decision points of the problem**
- The **domain**: Each variable x_i has a specific domain D_i , which is the set of valid values it can take.
- A **list of constraints** ($\mathbf{C}_1, \dots, \mathbf{C}_m$):
 - Constraints define permissible relationships between variables.
 - Each constraint C_j involves a subset of variables ($x_{j1}, \dots, x_{jk}$) and allows only valid assignments from the corresponding domains, represented as a relation $R_j \subseteq D_{j1} \times \dots \times D_{jk}$.
- **Objective Function (f)**
 - A function that maps the decision variables to a real-valued objective.
 - The goal is to maximize or minimize this function while satisfying the constraints.

$$\mathbf{f} : \mathbf{D}_1 \times \dots \times \mathbf{D}_n \rightarrow \mathbb{R}$$

Example: Linear Programming (Maximizing Profit in Production)

Variables:

x_1 = Number of product A units produced

x_2 = Number of product B units produced

Domain:

$x_1, x_2 \geq 0$ (Non-negative production quantities)

Constraints:

Resource limitations:

$2x_1 + 3x_2 \leq 100$ (Material constraint)

$4x_1 + x_2 \leq 80$ (Labor constraint)

Objective Function (Profit Maximization):

$$f(x_1, x_2) = 5x_1 + 7x_2$$

The goal is to find (x_1, x_2) that maximizes profit while satisfying resource constraints.

Preliminaries

The problem then consists of finding appropriate values $v_1, \dots, v_n$ for the variables, such that all of the following holds:

1. Feasibility

- Each selected value v_i must belong to its respective domain D_i .

$$v_i \in D_i \text{ for each } i = 1, \dots, n.$$

2. **Constraint Satisfaction:**

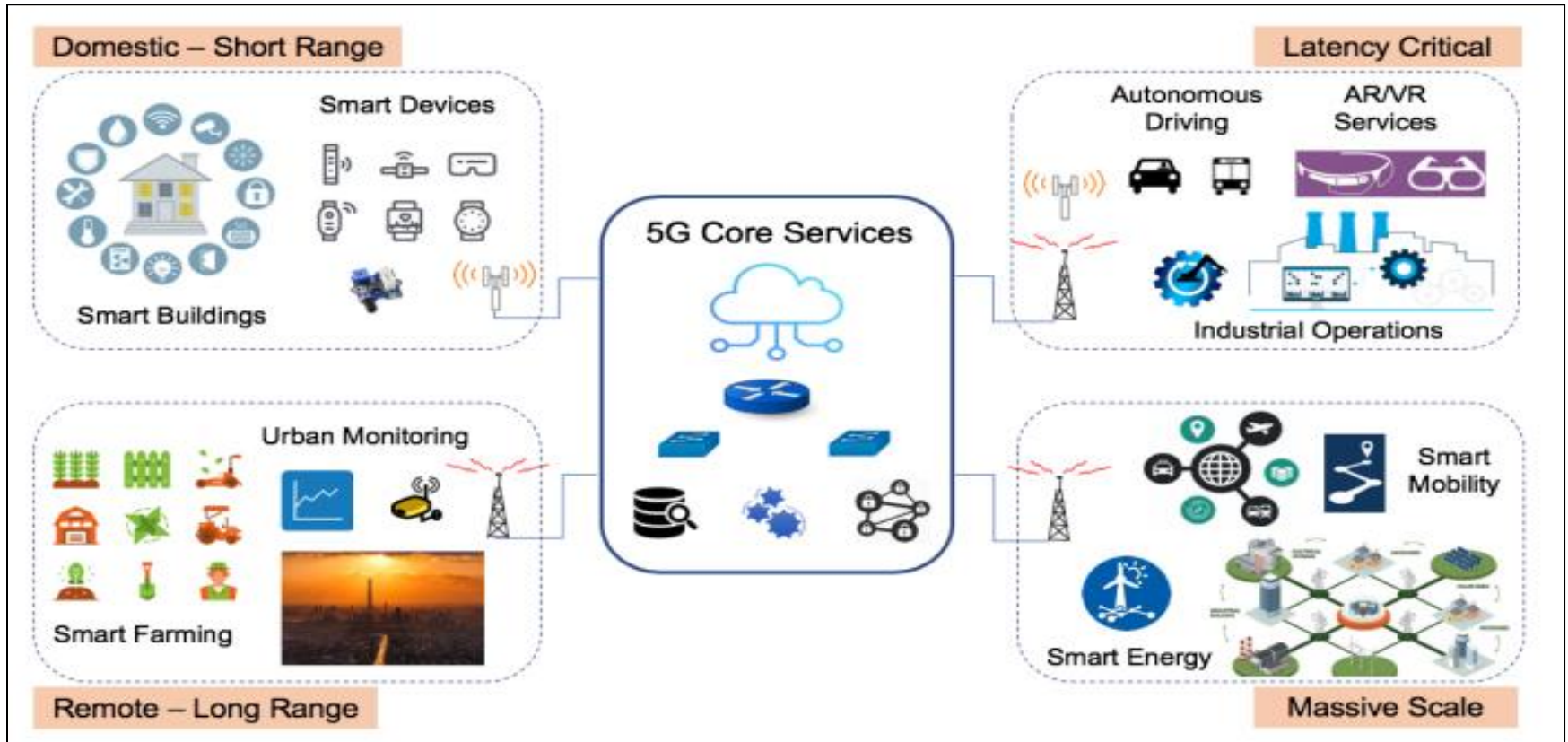
- The chosen values $(v_{j1}, \dots, v_{jk})$ must satisfy all constraints C_j .

$$(v_{j1}, \dots, v_{jk}) \in R_j$$

- (3) $f(v_1, \dots, v_n)$ is maximum among all $(v_1, \dots, v_n)$ tuples that satisfy (1) and (2).

The Case for Optimization in Fog Computing

- Fog computing can be seen as an extension of cloud computing towards the network edge, with the aim of **providing lower latencies** for latency-critical applications within end devices.

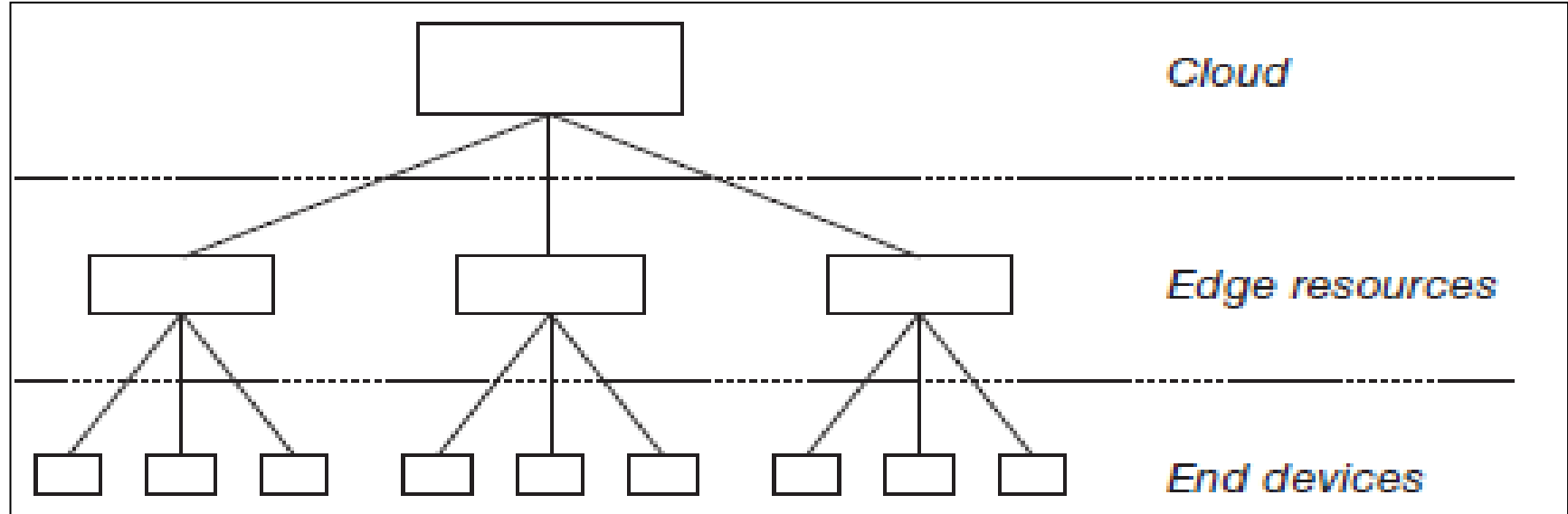


- The optimization objective of **minimizing latency** is a major driving force behind fog computing.
- Fog computing ***boosts the computing power of end devices***, allowing them to handle complex tasks quickly without using much energy.
- **Optimization** relating to **execution time** and **energy consumption** are also fundamental aspects of fog computing.
- Key areas for optimization

Resource Allocation Energy Efficiency Network Bandwidth Management Latency Reduction Task Scheduling Security and Privacy
--

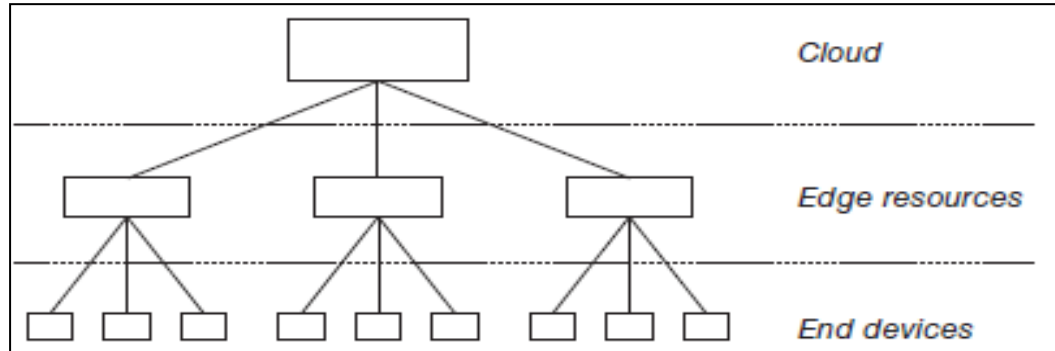
Formal Modeling Framework for Fog Computing

- Fog computing can be represented by a **hierarchical three-layer model**



- Higher layers** represent *higher computational capacity*, at the same time also *higher distance* – and thus *higher latency* – from the end devices.
- At the top layer, the cloud provides vast, high-performance resources that are cost-effective and energy-efficient.

- **Middle layer** consists of a **set of edge resources**: machines offering compute services near the network edge, e.g. in base stations, routers, or small, geographically distributed data centers of telecommunication providers.
- The edge resources are all connected to the cloud.
- **Lowest layer** contains the **end devices** like mobile phones or IoT devices.
- Each end device is connected to one of the edge resources.



- Let $\mathbf{c}$ denote the **cloud**
 $\mathbf{E}$ the **set of edge resources**
 $\mathbf{D}_e$ the **set of end devices connected to edge resource $e \in \mathbf{E}$**
 $\mathbf{D} = \bigcup_{e \in \mathbf{E}} \mathbf{D}_e$ the **set of all end devices**.

Cloud ($\mathbf{c}$): A data center.

Edge Resources ($\mathbf{E}$): Small servers in weather stations (e_1, e_2).

End Devices ($\mathbf{D}$): Sensors measuring soil moisture (d_1, d_2 connected to e_1 , d_3 connected to e_2).

- The set of all resources is

$$\mathbf{R} = \{ \mathbf{c} \} \cup \mathbf{E} \cup \mathbf{D}.$$

- Each resource $\mathbf{r} \in \mathbf{R}$ is associated with
 - a **compute capacity** $\alpha(\mathbf{r}) \in \mathbb{R}^+$ (Maximum amount of computation a resource $\mathbf{r}$ can handle.)

The cloud might have a compute capacity of 100 (arbitrary units), an edge resource 20, and an end device 1.

- a **compute speed** $\mathbf{s}(\mathbf{r}) \in \mathbb{R}^+$
- Each resource has some power consumption, which depends on its computational load.

- The **power consumption** of resource r increases by $\mathbf{w(r)} \in \mathbb{R}^+$ for every instruction to be carried out by r .

The cloud might have a power consumption increase of 0.1 units per instruction, an edge resource 0.05, and an end device 0.01.

- The **set of links (L)** between resources is

$$\mathbf{L} = \{\mathbf{ce} : \mathbf{e} \in \mathbf{E}\} \cup \{\mathbf{ed} : \mathbf{e} \in \mathbf{E}, \mathbf{d} \in \mathbf{De}\}.$$

L is made up of links between the cloud and edge resources (ce), plus links between edge resources and their connected end devices (ed).

- Each link $l \in L$ is associated with a

- **latency** $\mathbf{t(l)} \in \mathbb{R}^+$

Link between an edge resource and the cloud might have a latency of 10 milliseconds, while the link between an edge resource and an end device might be 2 milliseconds.

- **bandwidth** $\mathbf{b(l)} \in \mathbb{R}^+$

Link between the cloud and an edge resource might have a bandwidth of 1 Gigabit per second, while the link between an edge resource and an end device might be 100 Megabits per second.

- Power Consumption per Byte** : Transmitting one more byte of data over link l increases power consumption by $\mathbf{w(l)} \in \mathbb{R}^+$.

Notation	Explanation
c	cloud
E	set of edge resources
D_e	set of end devices connected to edge resource $e \in E$
R	set of all resources
$a(r)$	compute capacity of resource $r \in R$
$s(r)$	compute speed of resource $r \in R$
$w(r)$	marginal energy consumption of resource $r \in R$
L	set of all links between resources
$t(l)$	latency of link $l \in L$
$b(l)$	bandwidth of link $l \in L$
$w(l)$	marginal energy consumption of link $l \in L$

Metrics

- Regardless of the application or problem type, certain key metrics are important in fog computing.

1. Performance

- Performance is related to the *amount of time* needed to accomplish a certain task.
- In a fog computing setup, completing a task often requires using multiple resources, which may be spread across different levels of the reference model.
- The *completion time* of the task may depend

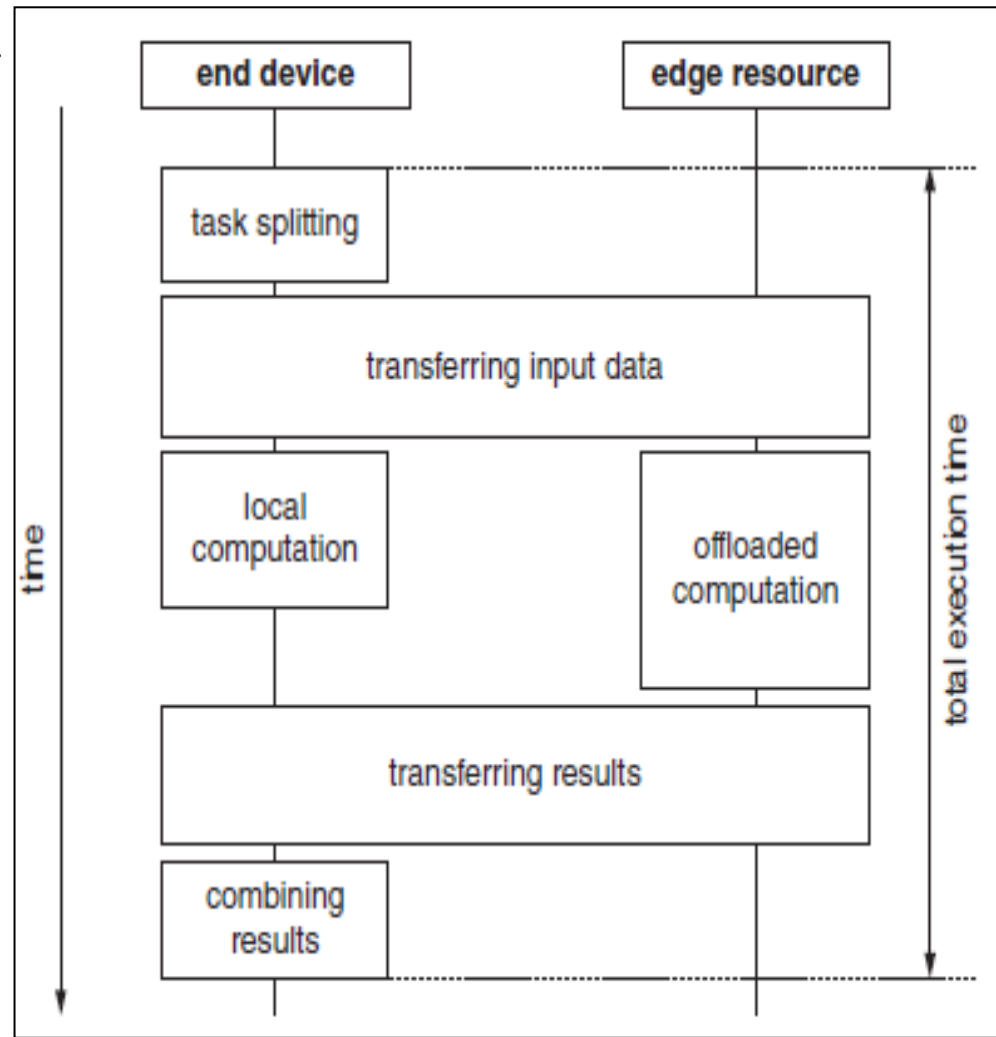
Completion time = Computation time of multiple resources + Time for data transfer between the resources.

- Some of these steps might be made in **parallel** (e.g., multiple devices can perform computations in parallel), whereas others must be made **one after the other** (e.g., the results of a computation can only be transferred once they have been computed).

- The **total execution time** depends on the critical path of compute and transfer steps.
- If a computation is split between an end device and an edge resource, the total execution time depends on both the processing time and the data transfer time between them.

Local Computation: The end device performs computation on the portion of the task it retained.

Offloaded Computation: The edge resource performs computation on the data it received from the end device.



Imagine you have a photo on your phone (end device) that you want to enhance using a powerful image editing software. This software requires more processing power than your phone has.

Task Splitting:

- **You (end device):** Decide which parts of the editing to do yourself (like cropping) and which parts to send to a more powerful computer (edge resource, like a nearby laptop or server).

Transferring Input Data:

- **You (end device):** Send the photo to the edge resource (laptop).

Local Computation:

- **You (end device):** While the photo is being enhanced on the laptop, you might do some minor edits on your phone (like adjusting brightness).

Offloaded Computation:

- **Laptop (edge resource):** Receives the photo and performs the complex enhancements using the image editing software.

Transferring Results:

- **Laptop (edge resource):** Sends the enhanced photo back to your phone.

Combining Results:

- **You (end device):** Receive the enhanced photo and combine it with the minor edits you made on your phone (the brightness adjustment).

The total time to get the final result is the sum of all the steps: splitting, transferring, local computation, offloaded computation, transferring back, and combining.

In a **smart healthcare monitoring system**, wearable devices (end devices) continuously track a patient's vital signs, such as heart rate, blood pressure, and oxygen levels. These wearables have limited processing power, so they offload complex computations to an edge server for faster analysis.

Task Splitting: The wearable device collects raw sensor data and decides which computations can be performed locally and which need to be offloaded.

Transferring Input Data: The device sends high-frequency, real-time sensor data to a nearby edge resource (e.g., a hospital server or a fog node).

Local Computation: The wearable performs simple calculations, such as averaging heart rate readings.

Offloaded Computation: The edge resource processes more complex tasks, such as detecting arrhythmias using AI-based models.

Transferring Results: The edge resource sends the processed results (e.g., abnormal heart patterns detected) back to the wearable.

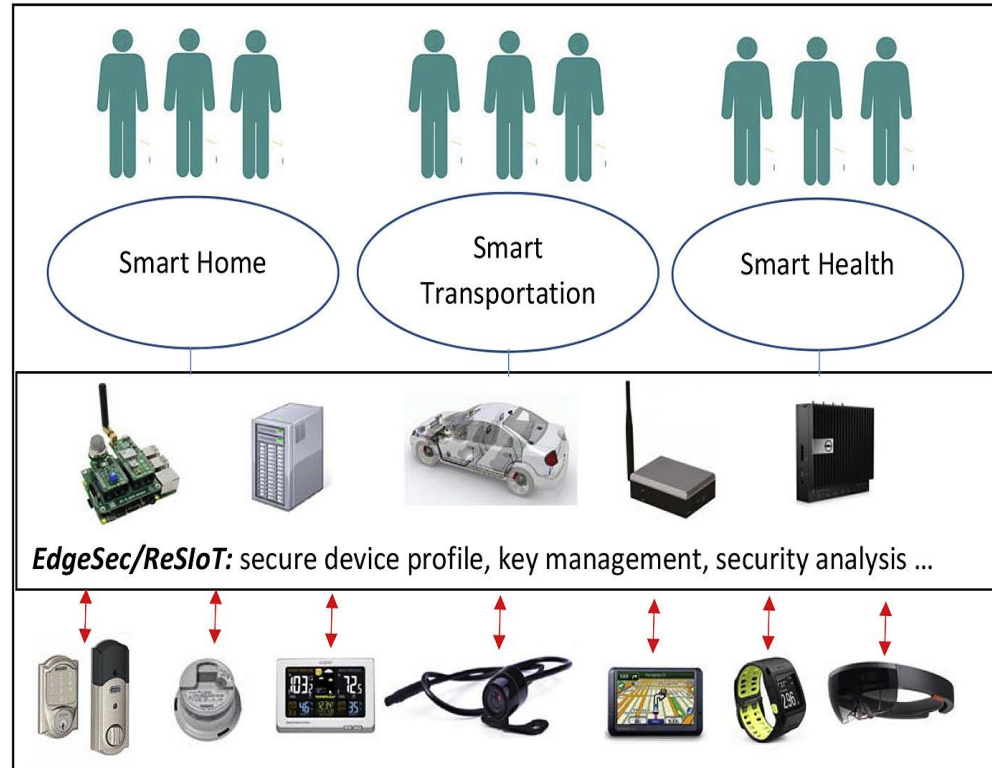
Combining Results: The wearable device integrates local and edge results to provide the patient with real-time feedback.

2. Resource Usage

- This particularly applies to end devices, which typically have **very limited CPU** and **memory capacity**.
- Edge resources usually have more power but can still be limited since devices like routers have restricted computing abilities.
- Higher CPU usage can slow down execution, meaning the application keeps running but takes longer.
 - This may be acceptable for some applications, but not for **time-critical** ones.
 - Running out of memory is a bigger problem than running out of other resources, because it can cause applications to crash.
- Beyond CPU and memory, also **network bandwidth** can be a scarce resource, both between end devices and edge resources and between edge resources and the cloud.
 - The use of network bandwidth may have to be either minimized or constrained by an upper bound.

3. Energy Consumption

- Energy is consumed by all resources as well as the network.
- Even **idle resources** and **unused network elements** consume energy, but their energy consumption increases with usage.
- Energy consumption is important on each layer of the fog, but in different ways.
- For **end devices**, **battery power** is often a bottleneck, and thus preserving it as much as possible is a primary concern.
- **Edge resources** are typically not battery-powered; hence, their energy consumption is less important.
- Overall energy consumption of the whole fog system is important because of its environmental impact.



4. Financial Costs

- Energy consumption has implications on financial costs.
- The **use of the cloud** or **edge infrastructure** may incur costs.
- These costs can be **fixed** or **usage-based**, or some combination.
- The **use of the network** for transferring data may incur costs.

Consider a smart irrigation system for farming that uses IoT sensors to monitor soil moisture and weather conditions.

Cloud Usage Costs: If data is processed and stored in the cloud, the farmer may pay a monthly subscription fee (fixed cost) or be charged based on the amount of data processed (usage-based cost).

Edge Computing Costs: If data is processed locally on an edge device (e.g., a small computer on the farm), there may be a one-time hardware cost but lower ongoing expenses.

Network Costs: Sending sensor data to the cloud via the internet may increase data transmission costs, especially if using a cellular network.

By optimizing where and how data is processed, the farmer can reduce both energy usage and financial costs.

5. Further Quality Attributes

- For the quality attributes the mentioned has to be taken into account

Reliability

- Achieved by creating *redundancy* in the architecture

Security

- Achieved by using appropriate *cryptographic* techniques for encryption

Privacy

- Achieved by applying *anonymization* of personal data

- There are several ways to address **quality attributes** during optimization of a fog system, as shown by the following representative examples
- (i) **To increase reliability**, multiple resources can perform the same critical task at the same time.
 - This ensures the result is available even if some resources fail and helps detect errors by comparing the results.
- **The higher the number of resources used in parallel, the higher level of reliability can be achieved this way.**
- Therefore, the number of resources used in parallel is an important optimization objective that should be maximized.
- (ii) **Both security and privacy** risks can be reduced by using trusted resources.
- Systems can use **reputation scores** to measure trust and prioritize reliable resources, ensuring safer and more efficient usage.

Reputation Score

Good

30 days ago: 544 pts ↑ 6% (68 pts)

YOUR SCORE

612



Locations

Goddard



★★★★★ 881

Competitors

Holly Sprints



★★★★★ 465

Kiddle Academy



★★★☆☆ 343

Goddard West



Brookfield South



Brookfield North



What has impacted your Reputation Score?

30 day report

Positive impact

Social Engagement ↑ < 1 pt

Review Length ↑ < 1 pt

Negative Impact

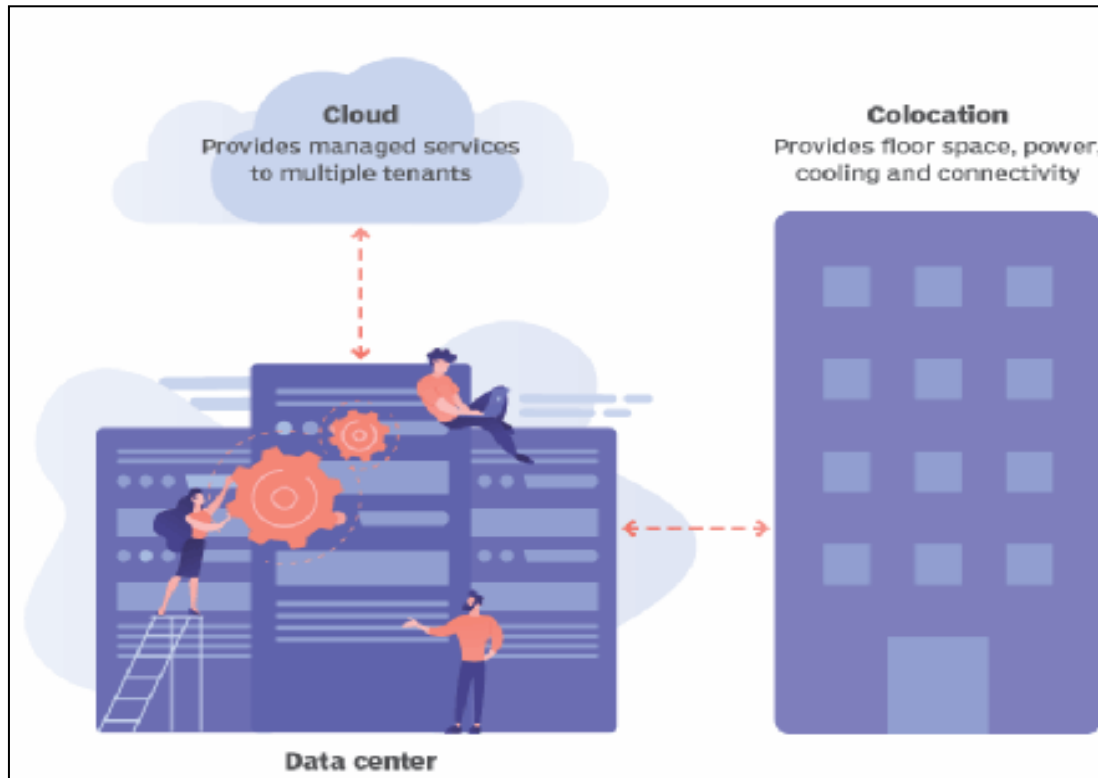
Star Average ↓ < 4 pt

Listing Accuracy ↓ < 1 pt

Review Response ↓ < 1 pt

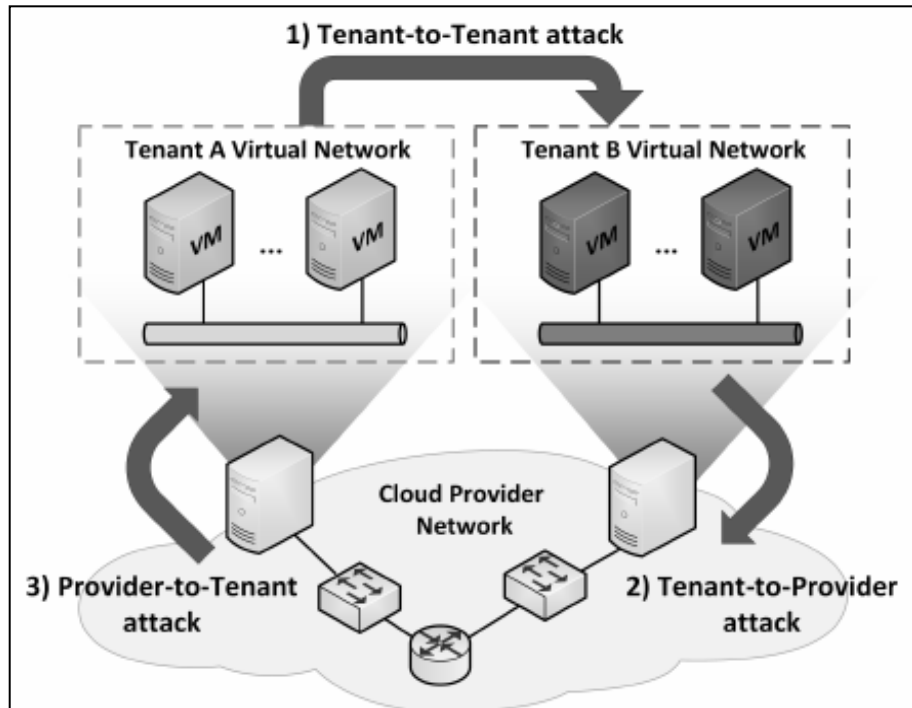
(iii) Co-location of computational tasks belonging to different users / tenants may increase the possibility of tenant-on-tenant attacks.

- Therefore, minimizing the number of tenants whose tasks are co-located is an optimization objective that helps to keep security and privacy risks at an acceptably low level.



Colocation, the placement of several entities in a single location. Colocation centre, a data center where companies can rent equipment, space, and bandwidth for computing services, known as colocation services.

- A **tenant** is a group of users with shared access and specific permissions in a software system.
- A tenant is the most fundamental construct of a **SaaS environment**.
 - In a **private cloud**, tenants are different teams or departments(within the same company.
 - In a **public cloud**, tenants are separate organizations sharing server space securely.



(iv) Co-location of tasks means running tasks from the same user on the same server or close together.

- This reduces the need to send data over the network, lowering the risk of cyberattacks like
 - Eavesdropping
 - man-in-the-middle attacks
 - other network threats.

Minimizing Resource Usage → Lower Security Risks

- Reducing the number of servers and storage used limits potential attack points for hackers.

If a company stores customer data in only one secured database instead of multiple unnecessary copies, it reduces the risk of data breaches.

Increasing Redundancy → Better Reliability but Higher Costs

- Keeping backup systems helps in case of failures, but it requires more resources and money.

A cloud provider having duplicate servers in multiple locations ensures uptime but increases operational costs.

Choosing Reputable Providers → Higher Security but Higher Costs

- Well-known service providers offer strong security measures but charge more.

Using AWS (Amazon Web Services) ensures data protection but costs more than a smaller, less-known provider.

Limiting Co-location → Better Privacy but Possible Performance Issues

- Not sharing server space with others enhances privacy but may reduce efficiency.

A bank using a private cloud instead of a shared one keeps customer data safer but may have higher latency and power usage.

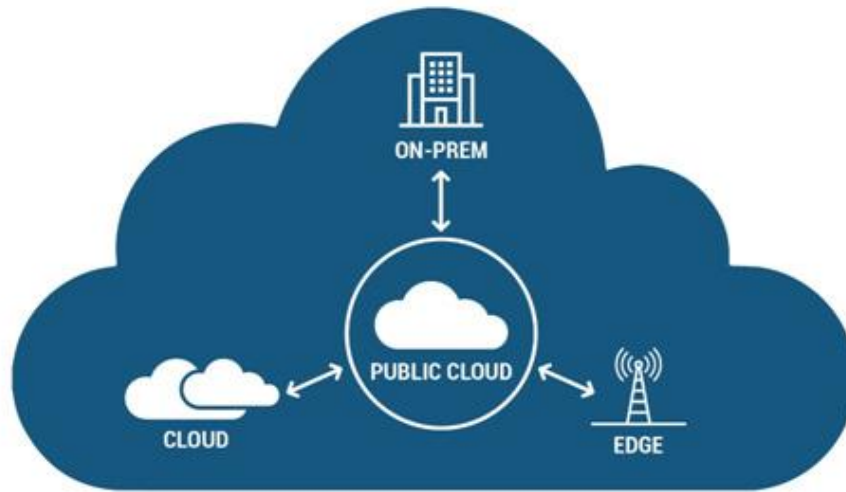
Optimization Opportunities along the Fog Architecture

- Optimization problems in fog computing can be classified based on the **three-layer fog model**:
- Real fog computing problems involve at least two layers.
- This consideration leads to the following classification of optimization problems in fog computing:

(i) Problems involving the cloud and the edge resources.

- This setup helps optimize energy use in cloud and edge resources while ensuring capacity and low latency.
- This setup is similar to *distributed cloud computing*.
- A key difference is that there are usually many more edge resources than data centers in a distributed cloud.

Consider a video streaming service. The main cloud data centers store and process large amounts of content, but edge servers (located closer to users) handle local streaming requests. This setup reduces delays and improves performance by distributing computing tasks between the cloud and the edge.

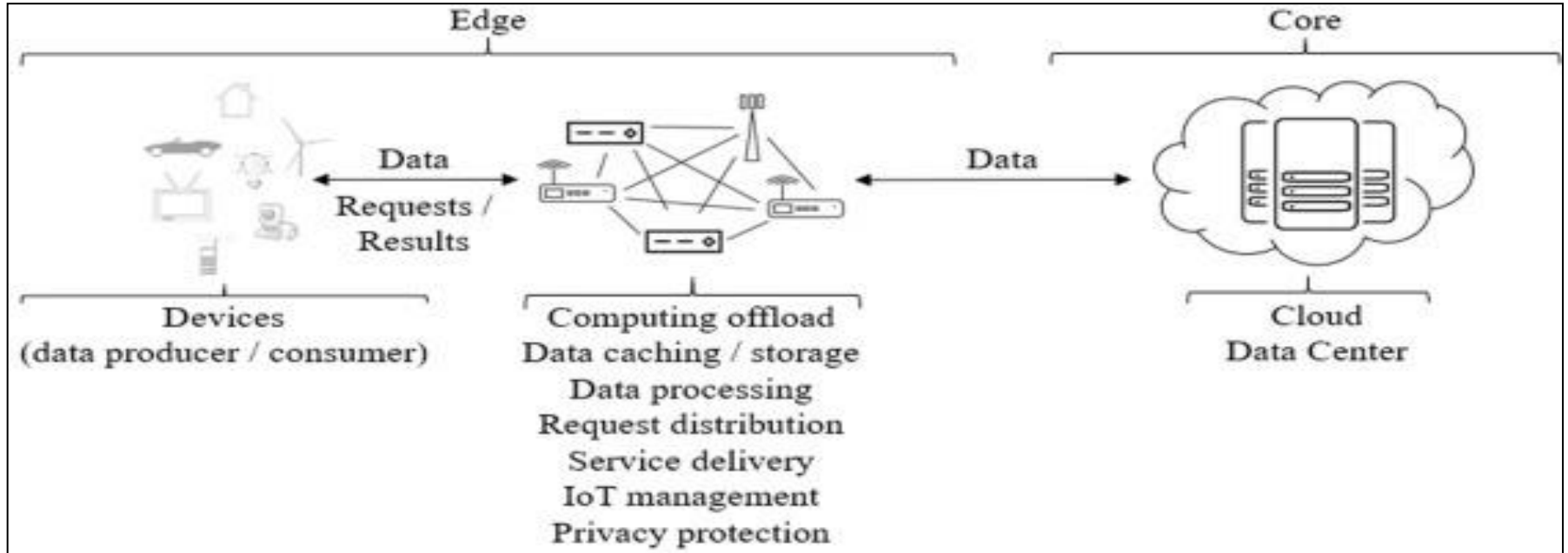


(ii) Problems involving edge resources and end devices

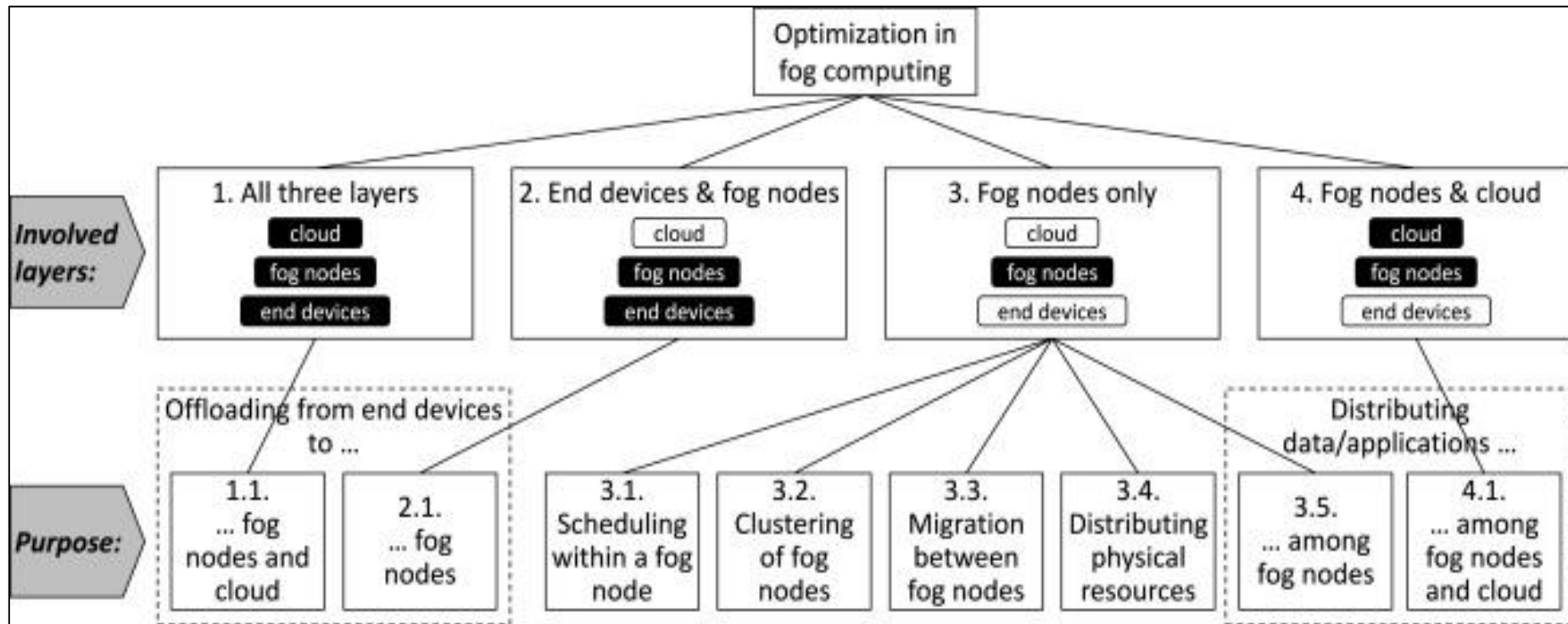
- End devices work with edge resources to share tasks, like offloading computations.
- This is common in fog computing, where optimization is important due to the limited power of end devices.

A smartphone using cloud gaming can offload heavy graphics processing to a nearby edge server. This reduces the phone's workload, saves battery, and ensures smooth gameplay.

Computation offloading means moving heavy tasks from an end device to powerful systems like the cloud or nearby edge devices.



- All three layers can be optimized together.
- One challenge is solving complex optimization problems that involve managing many fog resources and their decisions.



- Combining different optimization needs for the cloud, edge resources, and end devices into one problem is difficult because each has unique technical challenges.
- Different stakeholders update the cloud, edge resources, and end devices at different times. This is why each fog layer is optimized separately.

Imagine a hospital using wearable sensors to continuously monitor patients' vital signs (heart rate, blood pressure, etc.). This generates a lot of data that needs to be processed quickly and efficiently.

End Devices: These are the wearable sensors on the patients, collecting the raw vital signs data.

Fog Nodes: These could be small servers located within the hospital, perhaps on each floor or in specific departments. They are closer to the data source than the cloud.

Cloud: This is a larger, centralized data center (potentially off-site) with more powerful processing and storage capabilities.

1. All Three Layers Involved (End devices, Fog nodes, Cloud):

Initial Processing: The wearable sensors (end devices) might do some basic filtering of the data (e.g., removing noise).

Immediate Alerts: The fog nodes analyze the data for critical events like sudden drops in blood pressure. If detected, an immediate alert is sent to the nurses' station.

Long-Term Analysis: Less urgent data is sent to the cloud for long-term analysis, such as identifying trends in a patient's vital signs over several days. This could help doctors adjust treatment plans.

2. End Devices & Fog Nodes:

Local Processing: The fog nodes handle most of the data processing and analysis. For instance, they might track a patient's heart rate variability throughout the day.

Limited Cloud Use: Only summarized or aggregated data is sent to the cloud, reducing bandwidth usage. This could be daily summaries of patient vital signs.

3. Fog Nodes Only:

Isolated Network: In a scenario where network connectivity is limited (e.g., during a disaster), the fog nodes could operate independently. They might store a certain amount of patient data locally and perform all necessary analysis and alerting. This ensures continuous monitoring even with network disruptions.

4. Fog Nodes & Cloud:

Scalability: The cloud is used to provide additional storage and processing power as needed. For example, if the hospital suddenly needs to monitor a large number of patients during an emergency, the cloud can help scale up the fog network.

Backup & Redundancy: Patient data is backed up to the cloud for disaster recovery and to ensure data availability even if local fog nodes fail.

Scheduling within a fog node (3.1): Prioritizing which data to process first based on urgency (e.g., analyzing critical vital signs before less urgent data).

Clustering of fog nodes (3.2): Grouping fog nodes to work together to handle data from a specific area of the hospital (e.g., all nodes on the cardiology floor).

Migration between fog nodes (3.3): Moving data or tasks from one fog node to another to balance the workload or if a node fails.

Distributing physical resources (3.4): Allocating resources like processing power and memory efficiently across the fog network.

Distributing data/applications among fog nodes (3.5 & 4.1): Ensuring data and applications are available where they are needed, either within the fog network or between fog and cloud, to minimize latency and improve performance.

- In each of the fog layers, optimization may target the **distribution of data, code, tasks**, or a **combination of these**.
- In ***data-related optimization***, decisions have to be made about which pieces of data are stored and processed in the fog architecture.
- In ***code-related optimization***, program code can be deployed on multiple resources and the goal is to find the optimal placement of the program code.
- In ***task-related optimization***, the aim is to find the optimal split of tasks among multiple resources.

Optimization Opportunities along the Service Life Cycle

- Fog computing is characterized by the delivery and consumption of services.
- The different optimization opportunities at the different stages of the service life cycle, can be differentiate between the following options:

a. Design-time optimization

b. Deployment-time optimization

c. Run-time optimization

a. Design-time optimization

- When designing a fog service, exact details about the specific end devices are usually unknown.
- Therefore, optimization mainly focuses on the cloud and edge layers, where more information is available at the design stage.
- End-device optimization is limited to handling different types of devices, not specific device instances, which are only known at runtime.

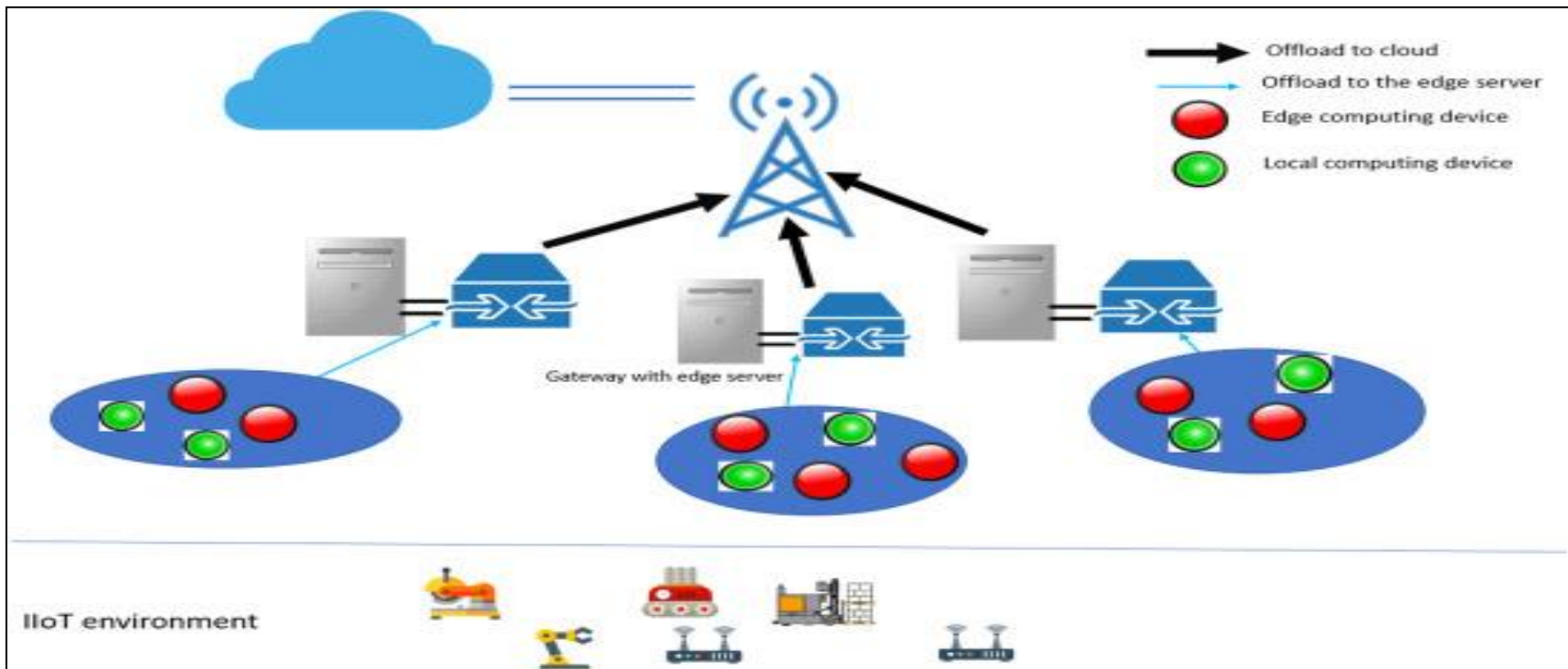
When building a smart home system, you may not know the exact brand of smart bulbs users will have, but you know they will use smart bulbs. So, you design the system to work with any smart bulb instead of focusing on a specific model.

b. Deployment-time optimization

- When planning to deploy a service on specific resources, the available resource information can help optimize the deployment for better performance.
- For example, the exact capacity of the edge resources to be used may become available at this time, so that the split of tasks between the cloud and the edge resources can be (re-)optimized.

c. Run-time optimization.

- Some aspects of a fog system can be optimized beforehand, during design or deployment. However, many important factors only become clear once the system is in operation.
- In a smart traffic management system, initial optimizations like server placement and data flow design can be done during deployment. However, real-time factors like sudden traffic congestion or network delays can only be addressed dynamically while the system is running.
- These aspects are vital for making sound optimization decisions.

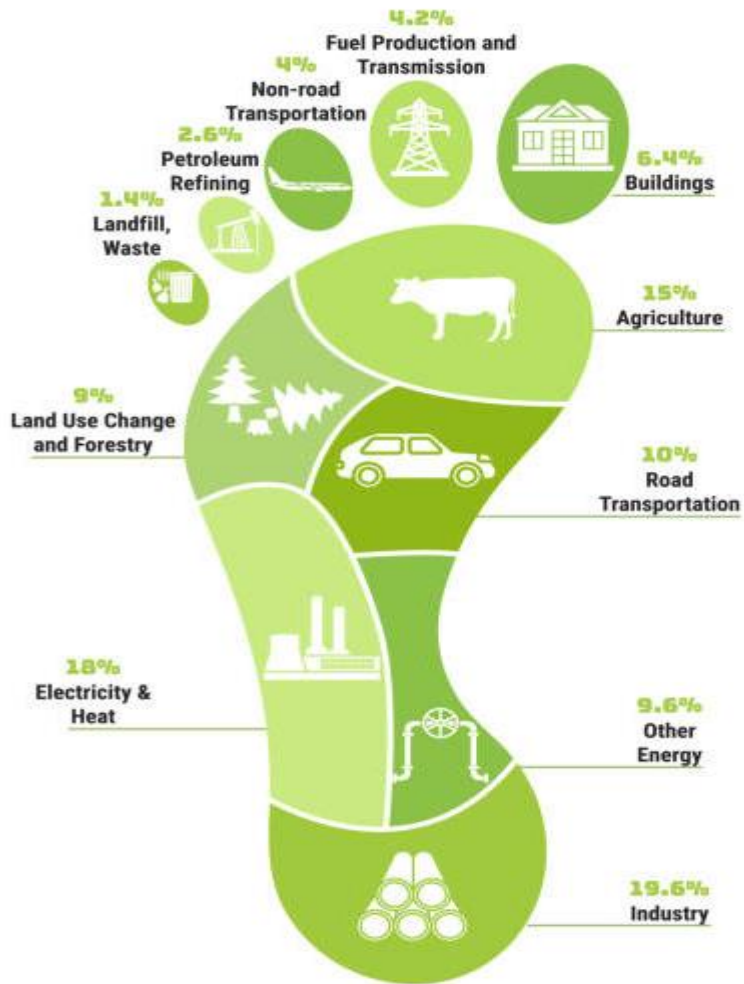


- These aspects keep changing during the operation of the system.
- Much of the system operation needs to be optimized during run time.
- This requires **continuous monitoring of important system parameters, analysis of whether the system still operates with acceptable effectiveness and efficiency, and re-optimization whenever necessary.**

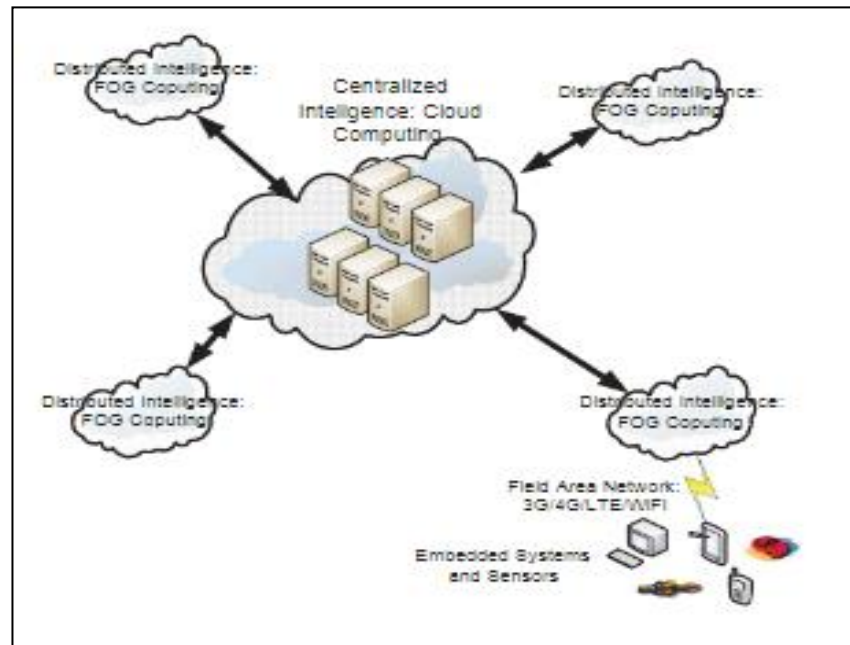
Toward a Taxonomy of Optimization Problems in Fog Computing

- Table shows the classification of the work of Do et al.

Paper:	Do et al.: A proximal algorithm for joint resource allocation and minimizing carbon footprint in geo-distributed fog computing [21]
Context / domain:	Video-streaming service with a central cloud serving distributed edge resources which, in turn, serve end devices
Considered metrics:	• “Utility” (weighted data rate of the edge resources) • Compute capacity of the cloud data center • Energy consumption of the cloud data center
Considered layer / resources:	• Cloud • Edge resources
Phase in life cycle:	Design / deployment time
Optimization algorithm:	Distributed iterative improvement

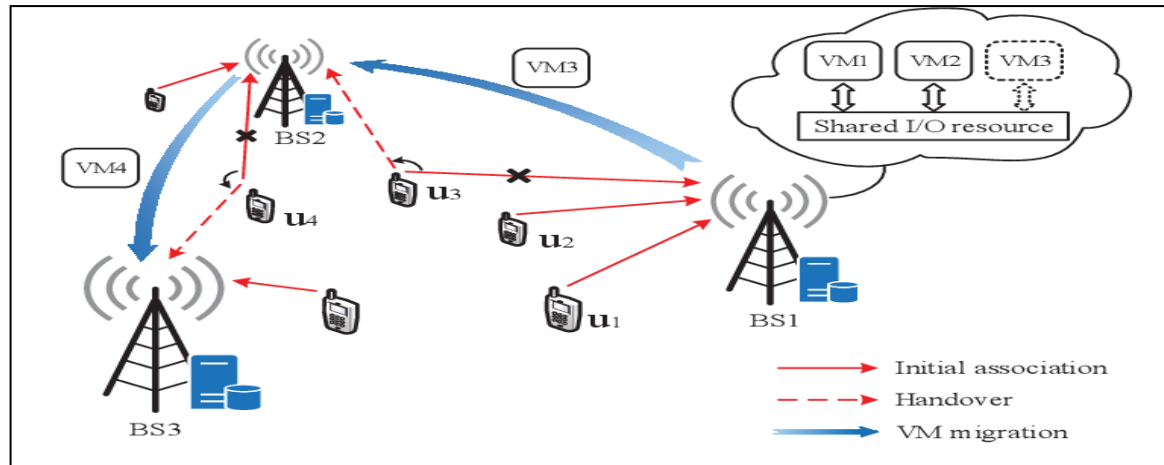


CARBON FOOTPRINT



- Table shows the classification of the work of Sardellitti et al.

Paper:	Sardellitti et al.: Joint optimization of radio and computational resources for multicell mobile-edge computing [22]
Context / domain:	Computation offloading from mobile end devices to an edge resource
Considered metrics:	• Energy consumption of the end devices • Total time to transfer and execute offloaded tasks • Amount of compute power of edge resource occupied by offloaded tasks of the devices
Considered layer / resources:	• Edge resource • End devices
Phase in life cycle:	Run time
Optimization algorithm:	Iterative heuristic using successive convex approximation



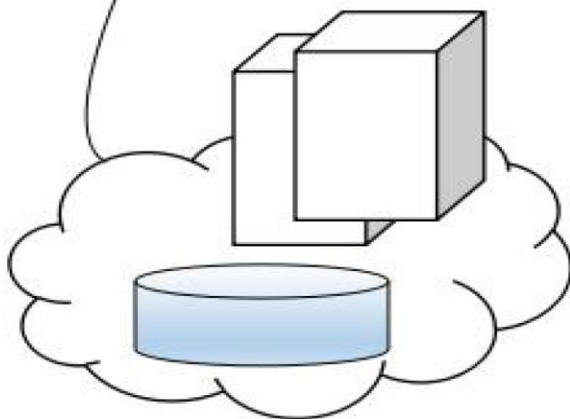


Computation resource

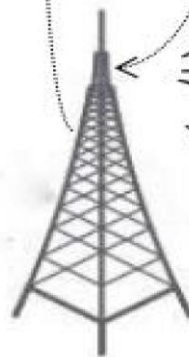
Tasks



Task offloading problem:
whether the task is offloaded to the edge cloud



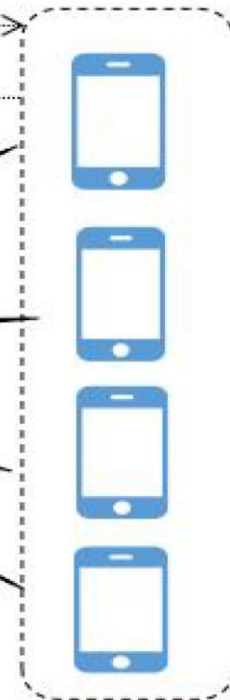
Edge cloud



Wireless Base-station

Results

Task offloading



Mobile devices

- Table describes the work of Mushunuri et al.

Paper:	Mushunuri et al.: Resource optimization in fog enabled IoT deployments [23]
Context / domain:	Cooperating mobile robots sharing compute tasks
Considered metrics:	• Communication cost between end devices and edge resource • Battery power of end devices • CPU capacity of end devices and edge resource
Considered layer / resources:	• Edge resource • End devices
Phase in life cycle:	Run time
Optimization algorithm:	Nonlinear optimization with the COBYLA (Constrained Optimization by Linear Approximations) algorithm within the NLOpt library